

固定流水线 Conservative SLA-Safe Evaluator v2.1

事件驱动、保守资源预留的固定 Layer-Level Pipeline 评价模型

1. Evaluator 的目标

给定一条已经确定的 Layer-Level 推理流水线 p , Evaluator 不尝试精确复现真实 LLM runtime, 而是利用:

- 固定流水线部署;
- 有限请求 workload;
- TTFT / TPOT SLA;
- GPU / 网络 profiling;
- 当前请求状态;

构造一个事件驱动、保守资源预留的状态评价过程。

Evaluator 最终返回:

$$E(p, W, SLA) = \underline{C}_p^{SLA}$$

其中 $\underline{C}_p^{SLA}$ 表示在给定 workload generation protocol 和 SLA 条件下, 固定流水线 p 按照本 Evaluator 的保守判定标准能够支持的最大 workload intensity。

它不是实际系统绝对最大 throughput, 也不表示无限时间排队系统意义下的严格稳定容量, 而是:

Conservative SLA-Safe Request Capacity

2. Evaluator 输入

2.1 固定流水线

p 包含:

- 每个连续 Layer Stage 部署在哪个节点;
- Stage 之间经过哪些物理链路;
- 节点显存容量;
- 节点硬件类型与计算性能参数;
- 链路带宽容量。

2.2 Workload

每个请求 r 至少包含:

$$r = (t_r^{arr}, L_r^{in}, L_r^{out})$$

其中:

- t_r^{arr} : 请求到达时间;
- L_r^{in} : 输入 token 数;
- L_r^{out} : 输出 token 数。

本 Evaluator 使用的是**有限请求轨迹**。即 workload 中的 Arrival 在有限时间后停止：

$$Arrival \rightarrow Arrival \rightarrow \dots \rightarrow Last\ Arrival.$$

最后一个请求到达以后，Evaluator 不立即结束，而是继续推进当前所有 active requests，直到：

$$\mathcal{A}(t_{end}) = \emptyset$$

即所有请求均 Finish。

因此一次完整评价过程为：

$$Finite\ Arrival \rightarrow Load\ Growth \rightarrow Peak \rightarrow Load\ Decline \rightarrow Drain \rightarrow Empty$$

若评价不同 workload 强度，可由基础 workload shape W_0 构造：

$$W_\lambda = \mathcal{G}(W_0, \lambda)$$

其中 λ 表示 workload intensity parameter，例如控制平均请求到达速率或压缩请求间隔。

因此这里的 λ 不要求具有无限时间 stationary arrival process 的含义。

2.3 SLA

每个请求具有：

$$\tau_r^{TTFT} \quad \text{以及} \quad \tau_r^{TPOT}.$$

TTFT 用于 Prefill 阶段约束；TPOT 用于 Decode 阶段各 context 区间的约束。

2.4 Compute Profiling

Evaluator 不要求必须拥有所有目标 GPU 并逐一进行真实测量。Compute Profiling 可以来自：

1. 实际 GPU profiling；
2. 公开 profiling 数据；
3. 基于模型结构与硬件参数建立的 analytical / synthetic profiling；
4. 使用公开测量结果校准后的估计模型。

最终 Evaluator 只要求 Profiling Layer 能提供近似接口：

$$T_P^{prof} = f_P(L^{in}, N_P, N_D, p, hardware)$$

以及：

$$T_D^{prof} = f_D(L_{ctx}, N_P, N_D, p, hardware).$$

其中：

- T_P^{prof} ：Prefill 计算时间；
- T_D^{prof} ：当前状态下 Decode 的近似 per-token compute time。

若缺少真实测量，可以基于：

$$\text{Model Architecture} + \text{GPU Compute Throughput} + \text{Memory Bandwidth}$$

建立近似性能模型，例如：

$$\hat{T} \approx \max \left(\frac{F}{\eta_c P_{peak}}, \frac{D}{\eta_m BW_{mem}} \right)$$

其中：

- F ：计算 FLOPs；
- D ：数据访问量；
- P_{peak} ：硬件峰值计算能力；
- BW_{mem} ：显存带宽；
- η_c, η_m ：由公开 profiling 或敏感性实验校准的有效效率系数。

因此本 Evaluator 使用的是：

Measured or Estimated Compute-Time Profiling

而不是要求 cycle-accurate GPU simulation。

3. 请求状态

请求主要处于 *Prefill* 或 *Decode*。Prefill 状态表示为：

$$P(L_r^{in}).$$

Decode 同时维护两类信息。

连续 Decode Progress

定义：

$$g_r(t) \in [0, L_r^{out}]$$

表示请求 r 当前已经完成的 **equivalent-token computational progress**。

它可以是连续值，例如：

$$g_r = 38.67.$$

该数值不表示真实系统生成了 0.67 个 token，而只是 Evaluator 内部用于记录 Decode 计算进度的连续状态变量。

离散 Decode Block State

设 Decode block 大小为 q ，则当前 block index 可由：

$$h_r(t) = \left\lfloor \frac{g_r(t)}{q} \right\rfloor$$

确定。

因此：

Continuous Progress $\neq$ Discrete Resource State

连续 progress 用于时间推进；Decode block 用于决定什么时候触发资源状态更新。

4. 事件驱动状态机

Evaluator 不采用固定时间步：

$$t, t + \Delta t, t + 2\Delta t, \dots$$

而只在系统发生有意义状态变化时更新。

定义四类事件：

$$\boxed{Arrival} \quad \boxed{P \rightarrow D} \quad \boxed{Decode \ Block \ Update} \quad \boxed{Finish}$$

系统状态演化为：

$$S_0 \xrightarrow{Event_1} S_1 \xrightarrow{Event_2} S_2 \rightarrow \dots$$

每次事件后执行：

$$\boxed{Update \ State \rightarrow Recompute \ Profiling \rightarrow Recompute \ Resource \ Requirement \rightarrow Check \ Red \ Lines \rightarrow Record}$$

5. Prefill 状态推进

Prefill 在第一版 Evaluator 中继续建模为：

Atomic Prefill

当请求 r Arrival 并进入 Prefill 时，根据：

- Prompt 长度；
- 当前 (N_P, N_D) ；
- Pipeline；
- Compute Profiling；

计算 $T_{P,r}$ ，并生成内部事件：

$$t_r^{P \rightarrow D} = t_r^{arr} + T_{P,r}.$$

在：

$$[t_r^{arr}, t_r^{P \rightarrow D})$$

期间，请求保持 Prefill 状态。

第一版模型中，Prefill 启动以后不因为新的 Arrival 对其内部 token 计算过程进行细粒度重放。

6. Decode 状态推进

Decode 使用：

Context Block Based Resource Update

同时使用：

Continuous Equivalent Decode Progress

负责时间推进。

6.1 Decode 当前推进速度

在某个系统状态 $S(t)$ 中共有 $N_P(t)$ 个 Prefill 请求和 $N_D(t)$ 个 Decode 请求。

通过 profiling 获得：

$$T_{D,r}^{token} = f_D(N_P, N_D, L_r^{ctx}, p, hardware).$$

在系统状态不发生事件变化的区间内，该值保持不变。

6.2 连续 Decode Progress

若两个事件发生在 t_k 与 t_{k+1} ，则 Decode 请求 r 的 equivalent progress 更新为：

$$g_r(t_{k+1}) = g_r(t_k) + \frac{t_{k+1} - t_k}{T_{D,r}^{token}(S_k)}$$

因此即使 Decode Block 尚未结束，外部 Arrival 也不会导致此前已经完成的 Decode 工作被丢失。

例如：

$$g_r = 32, \quad T_D^{token} = 3 \text{ ms/token}.$$

若 20 ms 后发生 Arrival，则：

$$g_r = 32 + \frac{20}{3} = 38.67.$$

若新的系统状态使：

$$T_D^{token} = 4 \text{ ms/token},$$

则距离 48-token block boundary 还有：

$$48 - 38.67 = 9.33$$

个 equivalent tokens。

于是新的 block event 时间为：

$$t_{next}^{DecodeBlock} = t + 9.33 \times 4 \text{ ms}.$$

6.3 Decode Block Update

设 block size 为 q 。当 g_r 跨越新的 block boundary 时：

$$hq \rightarrow (h+1)q$$

触发：

Decode Block Update

此时：

- 更新 Decode block index；
- 更新用于 profiling 和资源估计的 context 区间；
- 重新计算 SLA 网络资源需求；
- 重新计算 KV Cache 显存；
- 检查资源红线。

因此：

Progress is continuous, resource state is block-based.

6.4 Finish

当：

$$g_r(t) \geq L_r^{out}$$

时触发：

Finish

请求离开 active request set。

7. 下一事件选择

任意当前时刻 t ，Evaluator 比较：

外部事件

$$t_{next}^{Arrival}$$

内部事件

所有 active requests 中最近的：

$$t_{next}^{P \rightarrow D}, \quad t_{next}^{DecodeBlock}, \quad t_{next}^{Finish}.$$

因此：

$$t_{next} = \min \left\{ t_{next}^{Arrival}, t_{next}^{P \rightarrow D}, t_{next}^{DecodeBlock}, t_{next}^{Finish} \right\}$$

系统时间直接跳到 t_{next} 。

在真正执行 event batch 前，首先利用旧状态 $S(t^-)$ 推进所有 Decode 请求的连续 progress，然后执行事件。

若多个事件具有完全相同 timestamp，则作为同一个 event batch：

$$S(t^-) \xrightarrow{\mathcal{E}(t)} S(t^+)$$

原子更新。

不检查 batch 内由程序执行顺序产生的人工中间状态。

8. SLA $\rightarrow$ 网络最低资源需求

状态更新后，对 active requests 重新计算 SLA 资源需求。

对于请求 r ，首先计算 SLA 中留给网络的时间：

$$\Delta_r = \tau_r - T_{comp,r} - T_{queue,r} - T_{fix,r}$$

其中 $T_{comp,r}$ 直接由 profiling 得到。

Prefill 使用 τ_r^{TTFT} ；Decode 使用 τ_r^{TPOT} 。

若：

$$\Delta_r \leq 0$$

则状态直接判定：

Infeasible

其含义是：即使网络传输时间为 0，当前 compute / queue / fixed overhead 已经无法满足 SLA。
因此 GPU compute feasibility 已经通过：

$$Profiling \rightarrow T_{comp} \rightarrow SLA \text{ Time Budget}$$

进入 Evaluator。

需要注意：若 profiling 数据已经包含某些 GPU contention 或 scheduling delay，则这些延迟不能再次完整计入独立的 T_{queue} ，避免 double counting。

若：

$$\Delta_r > 0$$

则按照：

$$w_{e,r} = \frac{D_{e,r}/B_e}{\sum_j D_{j,r}/B_j}$$

分配网络时间预算：

$$\delta_{e,r} = w_{e,r} \Delta_r.$$

反推出最低链路服务能力：

$$b_{e,r}^{req} = \frac{D_{e,r}}{\delta_{e,r}}$$

这里 $b_{e,r}^{req}$ 不是 runtime 中请求瞬时实际获得的带宽，而是：

在当前请求状态和 SLA 条件下，链路 e 为该请求至少需要承诺的服务能力。

9. 保守网络资源记账

对于物理链路 e ：

$$B_e^{req}(t) = \sum_{r \in \mathcal{A}(t)} b_{e,r}^{req}(t)$$

要求：

$$B_e^{req}(t) \leq B_e$$

Evaluator 不利用：

- 精确通信错峰；
- 微观统计复用；
- activation 实际传完后的瞬时带宽释放。

定义：

$$H_e(t) = B_e - B_e^{req}(t)$$

表示当前剩余可承诺带宽。

10. Decode Context 增长导致的动态资源变化

随着 Decode block 更新：

$$h \rightarrow h + 1,$$

对应 context 区间增大。

重新查询：

$$T_{comp,r}^D = f_D(L_r^{ctx}, N_P, N_D).$$

通常：

$$L_r^{ctx} \uparrow \Rightarrow T_{comp,r}^D \uparrow.$$

因此：

$$\Delta_r = \tau_r^{TPOT} - T_{comp,r}^D - T_{queue,r} - T_{fix,r}$$

可能下降：

$$\Delta_r \downarrow.$$

进而：

$$\boxed{b_{e,r}^{req} \uparrow}$$

因此 Decode 请求即使早期可行，随着 context 增长，也可能在后续 Decode Block Update 中突破 SLA/network 红线。

11. 显存红线

Compute 不再采用独立的：

$$C_i^{use} \leq C_i$$

加和式红线。

Compute pressure 已通过 profiling 转化为 T_{comp} 。

但显存仍然需要显式容量记账。对节点 i ：

$$M_i^{use}(t) = M_i^{weight} + M_i^{KV}(t) + M_i^{workspace} + M_i^{margin}.$$

要求：

$$\boxed{M_i^{use}(t) \leq M_i}$$

其中：

- M_i^{weight} ：固定模型权重；
- $M_i^{KV}(t)$ ：所有 active requests 在该节点对应 Layer Stage 上的 KV Cache；
- $M_i^{workspace}$ ：运行时工作空间；
- M_i^{margin} ：安全余量。

KV Cache 随 N_{active} 以及 L_{ctx} 动态增长，因此在相关事件发生后需要重新计算。

12. Evaluator 作为状态记录器

Evaluator 不只返回最终可行 / 不可行结果，同时保存完整状态轨迹：

$$\mathcal{T}_p(W) = \{(t_k, S_k, R_k)\}_{k=0}^K$$

状态日志至少可以包含：

$$N_P(t_k), \quad N_D(t_k),$$

每个请求的：

$$g_r(t_k), \quad DecodeBlock_r(t_k), \quad L_r^{ctx},$$

每条链路的：

$$B_e^{req}(t_k), \quad H_e(t_k),$$

以及节点显存：

$$M_i^{use}(t_k).$$

若发生 violation，记录：

$$Violation = (t_k, type, object, required, capacity, system\ state)$$

例如可以记录：

- violation 发生时间；
- Network / Memory / SLA-time violation；
- 哪条链路或哪个 GPU 节点；
- 当前需求与容量；
- 当前 (N_P, N_D) ；
- active requests 的 Decode context 分布。

Evaluator 第一阶段不预先规定必须使用哪些高级统计指标。

后续可根据实验观察进一步提取：

- minimum bandwidth headroom；
- peak memory utilization；
- first violation state；
- dangerous context region；
- peak (N_P/N_D) ；
- violation trigger 类型。

13. 单个候选 Workload Intensity 的可行性判断

对于给定 λ ，由 workload generator 得到有限请求轨迹：

$$W_\lambda.$$

Evaluator 从第一个 Arrival 开始运行，一直到：

$$\mathcal{A}(t_{end}) = \emptyset$$

即所有请求全部完成。

在完整 trajectory 中检查：

SLA Compute-Time Feasibility

$$\Delta_r > 0$$

Network

$$B_e^{req}(t) \leq B_e$$

Memory

$$M_i^{use}(t) \leq M_i.$$

只要任意状态出现：

$$\Delta_r \leq 0$$

或：

$$B_e^{req} > B_e$$

或：

$$M_i^{use} > M_i,$$

则：

$$F_p(W_\lambda) = 0$$

否则：

$$F_p(W_\lambda) = 1.$$

这里的可行性指：

整个有限 workload trajectory 从开始、负载上升、峰值、下降直到所有请求 drain 完毕，都没有突破 Evaluator 的保守 SLA/resource red line。

它不声称证明无限时间 stochastic queue 的稳定性。

14. Evaluator 最终返回值

不断提高 workload intensity λ ，寻找最大的安全强度：

$$\underline{C}_p^{SLA} = \sup \{ \lambda : F_p(W_\lambda) = 1 \}$$

因此最终接口仍然为：

$$Evaluator(p, W, SLA) \rightarrow \underline{C}_p^{SLA}$$

其准确语义是：

固定流水线 p 在给定有限 workload generation protocol、TTFT / TPOT SLA、profiling model 和本 Evaluator 的保守资源承诺规则下，能够承受的最大 workload intensity。

15. Evaluator 的核心定位

本 Evaluator 不追求：

Exact Runtime Simulation

也不声称：

$$\underline{C}_p^{SLA} = C_p^{real,max}.$$

其定位是：

Low-cost Pre-deployment Conservative Capacity Estimation

整体流程为：

Fixed Pipeline → Finite Workload → Event-Driven State Evolution

→ Profiling-based Compute Time → SLA-derived Network Commitment

→ Network/Memory/SLA Red Lines → State Logging → Conservative SLA-Safe Capacity

其中：

- Compute 通过 profiling 转化为时间并进入 TTFT / TPOT budget；
- Network 通过 SLA 反推最低服务能力并进行保守加和；
- Memory 通过显式动态占用进行硬容量检查；
- Decode 使用连续 equivalent-token progress 推进时间，但只在 context block boundary 更新离散 Decode resource state；
- Workload 是有限完整轨迹，最后所有请求必须 drain 完毕；
- Evaluator 保存完整状态轨迹，为后续实验分析提供基础。

最终：

$$p \longrightarrow \underline{C}_p^{SLA}$$

可进一步作为 Layer-Level Deployment Search 中的固定流水线评价函数。